Universidad ORT Uruguay
Facultad de Ingeniería

Obligatorio 1

Entregado como requisito para la materia
Machine Learning para Sistemas Inteligentes

Repositorio de GitHub

Juan Manuel Reolon — 331598

Docentes: Sergio Fabián y Johny Kidd

2026

Índice

1. Preprocesamiento 3
   1.1. Análisis de calidad de datos 3
   1.2. Tratamiento de datos 3
2. Modelos 4
   2.1. Validación 4
   2.2. Árbol de Clasificación 4
      2.2.1. Descripción del modelo 4
      2.2.2. Modelo simple 4
      2.2.3. Ajuste del modelo 4
      2.2.4. Análisis 4
      2.2.5. Elección 5
      2.2.6. Gráficos complementarios 6
      2.2.7. Conclusiones 6
   2.3. Regresión Logística 6
      2.3.1. Descripción del modelo 6
      2.3.2. Modelo simple 6
      2.3.3. Ajuste del modelo 7
      2.3.4. Análisis 7
      2.3.5. Elección 7
      2.3.6. Gráficos complementarios 8
      2.3.7. Conclusiones 8
3. Selección de modelo final 8
   3.1. Comparación de modelos 8
   3.2. Justificación de elección final 8
4. Componente de investigación 8
5. Clasificación multiclase del genero 8
6. Feature Engineering 8
   6.1. Manejo de variables categóricas 8
   6.2. Creación de nuevas características 8
   6.3. Selección de características 8
   6.4. Análisis del impacto de variables 8

1. Preprocesamiento

Para este trabajo es necesario aplicar un preprocesamiento a los datos antes de entrenar los modelos. Los datos crudos suelen tener valores faltantes, errores de carga, escalas muy distintas entre variables y atributos que no aportan información útil o que la aportan de forma engañosa, lo que puede llevar al modelo a tomar decisiones equivocadas.

El preprocesamiento busca limpiar y transformar los datos para que reflejen de forma fiel la realidad, excluir aquellos atributos que no contribuyen a la predicción y representar cada variable en un formato que los modelos puedan aprovechar. De esta forma se busca obtener mejores resultados, modelos más robustos y una comparación justa entre los distintos enfoques evaluados.

1.1. Análisis de calidad de datos

Durante el análisis de calidad de los datos se identifican varias inconsistencias. En las variables budget y revenue, un 21% y un 29% de los registros tienen valor 0, lo que no representa un monto real sino la ausencia del dato. También se encuentran montos bajos (por ejemplo, películas con un presupuesto de 7 o de 110 dólares), que corresponden a errores de carga en otra unidad, probablemente en millones. Algo similar ocurre con runtime, donde algunas películas figuran con duración 0. Si estos valores se usaran tal cual, el modelo los interpretaría como películas de costo o duración nula y aprendería relaciones falsas, por lo que se decide tratarlos como valores faltantes.

budget	keywords	original_la	original_ti	overview	popularity	release_da	revenue	runtime	status	vote_cour	director	main_gen	opinion
70000000	budapest	en	I Spy	When the	13.26763	########	33561137	97	Released	269	Betty Thor	Action	0
7	flooding f	en	Split Secor	In a futuris	4.857028	5/1/1992	5	90	Released	63	Tony May	Thriller	0
14000000	college su	en	Gossip	On a beau	5.833687	########	5108820	90	Released	68	Davis Gug	Drama	0
15000000	barcelona	en	Vicky Crist	Two girlfri	32.75825	########	96408652	96	Released	1020	Woody All	Drama	1
2.5E+08	witch mag	en	Harry Pott	As Harry b	98.88564	7/7/2009	9.34E+08	153	Released	5293	David Yate	Adventure	1
70000000	saving the	en	AVP: Alien	When scie	42.95722	########	1.71E+08	101	Released	1217	Paul W.S.	Adventure	0
9000000	politics su	en	The Conte	Political th	7.029126	########	0	126	Released	60	Rod Lurie	Drama	1

Ejemplos de inconsistencias detectadas en el dataset

1.2. Tratamiento de datos

Además de los valores ya mencionados, se agrega para budget y revenue un indicador binario de “dato desconocido”, ya que la ausencia del dato resulta informativa por sí misma. En cambio, los 0 de vote_count se mantienen, porque representan un dato real: todas las películas sin votos tienen opinión negativa. Los faltantes se imputan con la mediana, por ser robusta a valores extremos, y no se elimina ningún registro, dado que el conjunto de test contiene casos similares que también deben predecirse.

Las variables muy sesgadas (budget, revenue, vote_count y popularity) se transforman con log(1 + x) para reducir el peso de los extremos, y todas las numéricas se estandarizan. De la fecha de estreno se extraen el año y el mes, y se crean las variables ROI, sin_keywords y cant_keywords.

El género y el idioma se codifican con one-hot, agrupando los idiomas poco frecuentes en “otros”. Para director, con casi 2000 valores distintos, se evalúan frequency encoding y target encoding frente a no utilizarlo. Se descartan status (prácticamente constante), original_title (casi un identificador) y overview (reservado para NLP).

2. Modelos

2.1. Validación

Todos los modelos se evalúan con validación cruzada estratificada de 5 folds. El conjunto de entrenamiento se divide en cinco partes que mantienen la proporción de cada clase, y cada modelo se entrena cinco veces, usando cuatro partes para entrenar y la restante para validar. Frente a un holdout simple, este esquema aprovecha todas las películas para entrenar y validar, y permite medir la estabilidad del resultado a través del desvío entre folds. Los folds son los mismos para todos los modelos, lo que asegura una comparación justa. Además, el preprocesamiento se incluye dentro del pipeline de cada modelo, por lo que la imputación, la estandarización y las codificaciones se ajustan solo con los datos de entrenamiento de cada fold, evitando que información de validación se filtre al modelo. En cada caso se registran accuracy, precision, recall, F1 y AUC, y se toma el AUC como métrica principal, ya que es la utilizada en la competencia de Kaggle y no depende del umbral de clasificación.

2.2. Árbol de Clasificación

2.2.1. Descripción del modelo

Un árbol de decisión clasifica cada película mediante una secuencia de preguntas sobre sus atributos. En cada nodo se elige la variable y el umbral que mejor separan las clases según una medida de impureza (Gini o entropía), y cada hoja asigna la clase mayoritaria de las películas que contiene. Se elige como primer modelo por ser fácilmente interpretable. Su principal debilidad es la tendencia al sobreajuste cuando crece sin restricciones.

2.2.2. Modelo simple

Como punto de partida se entrena un árbol con los hiperparámetros por defecto de scikit-learn, es decir, sin límite de profundidad y con hojas de una sola película. El modelo alcanza un AUC de 1,00 en entrenamiento pero de solo 0,667 en validación: el árbol memoriza los datos de entrenamiento y no logra generalizar, lo que confirma la necesidad de ajustar sus hiperparámetros.

2.2.3. Ajuste del modelo

Para ajustar el modelo se define un conjunto de valores posibles para tres hiperparámetros: la profundidad máxima del árbol (max_depth), la cantidad mínima de películas por hoja (min_samples_leaf) y el criterio de división (criterion). Mediante una búsqueda en grilla se entrenan y evalúan con validación cruzada todas las combinaciones posibles de estos valores, 108 en total. Así se analiza cómo influye cada hiperparámetro y cómo interactúan entre sí, para identificar la configuración que mejor generaliza.

2.2.4. Análisis

Una vez evaluadas las 108 combinaciones de la grilla, se analiza cuál presenta el mejor comportamiento. Como criterio principal se utiliza el AUC promedio en validación cruzada, ya que mide la capacidad del modelo para ordenar correctamente las películas en datos que no vio durante el entrenamiento. Además del valor medio, se observa su desvío entre folds, que refleja la estabilidad del modelo, y la diferencia entre el AUC de entrenamiento y el de

validación, que indica el grado de sobreajuste. Se selecciona la configuración con mayor AUC de validación, siempre que su diferencia con el entrenamiento sea reducida.

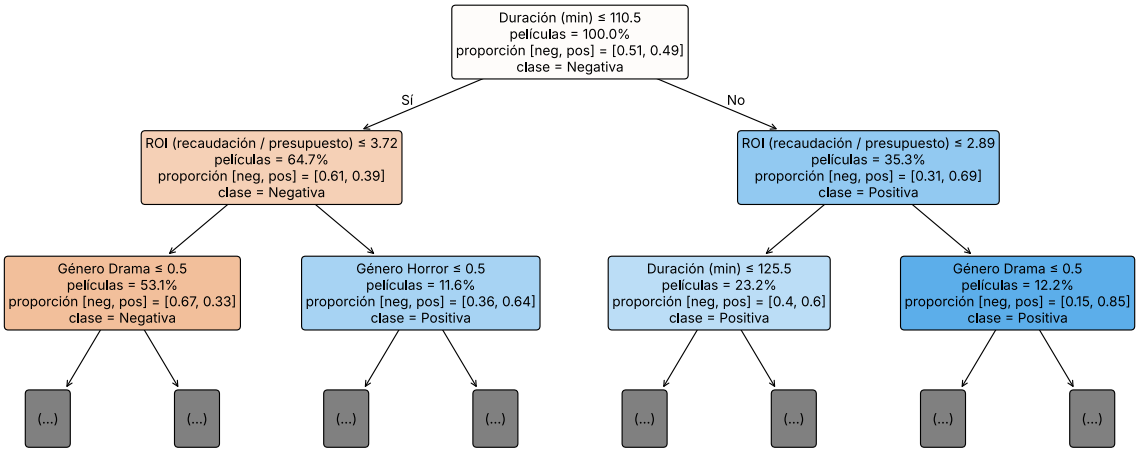
Profundidad máx.	Mín. por hoja	Criterio	AUC val.	AUC train
8	**50**	**gini**	**0,769 ± 0,013**	0,813
sin límite	50	gini	0,768 ± 0,012	0,819
15	50	gini	0,768 ± 0,012	0,819
10	50	gini	0,767 ± 0,010	0,817
6	50	gini	0,767 ± 0,009	0,803
sin límite	1	gini	0,667 ± 0,004	1,000

Mejores combinaciones de hiperparámetros del árbol y modelo simple (última fila)

Las cinco mejores combinaciones comparten un mínimo de 50 películas por hoja y sus diferencias de AUC son menores al desvío entre folds, por lo que el tamaño mínimo de hoja resulta el hiperparámetro determinante, mientras que la profundidad y el criterio tienen un efecto menor. Un árbol de profundidad 6 obtiene un resultado equivalente con una brecha algo menor respecto al entrenamiento, por lo que también sería una alternativa razonable.

2.2.5. Elección

Con estos criterios se selecciona un árbol con profundidad máxima de 8, un mínimo de 50 películas por hoja y el criterio gini, que alcanza un AUC de validación de 0,769 ± 0,013 con una diferencia reducida respecto al entrenamiento.



Primeros niveles del árbol elegido. Si la condición del nodo se cumple, se sigue la rama izquierda

La primera división separa las películas según su duración: entre las de más de 110 minutos predomina la opinión positiva (69%), mientras que entre las más cortas predomina la negativa. En ambas ramas la siguiente división se realiza por ROI, y las películas largas con ROI alto alcanzan un 85% de opiniones positivas.

La siguiente tabla compara el árbol elegido con el modelo simple e incluye el AUC obtenido en Kaggle.

Modelo	Accuracy	Precision	Recall	F1	AUC val.	AUC train	AUC Kaggle
Modelo simple	0,667	0,663	0,660	0,662	0,667	1,000	—
Modelo ajustado	0,697	0,705	0,669	0,685	**0,769**	0,813	**0,771**

Métricas del árbol elegido y del modelo simple

2.2.6. Gráficos complementarios

La matriz de confusión muestra que el modelo clasifica correctamente el 73% de las películas con opinión negativa y el 67% de las positivas. Los errores están relativamente balanceados, aunque los falsos negativos superan a los falsos positivos, por lo que el modelo tiende a ser algo conservador al predecir una opinión positiva.

Para no extender el informe, la matriz de confusión, el gráfico de sobreajuste según la profundidad y la importancia de las variables se encuentran en la notebook Arbol_Clasificacion.ipynb, incluida en la entrega.

2.2.7. Conclusiones

El árbol ajustado alcanza un AUC de validación de 0,769, una mejora de 0,10 respecto al modelo simple, y el AUC obtenido en Kaggle (0,771) es prácticamente igual, lo que indica que la validación cruzada estima de forma confiable el rendimiento sobre datos nuevos. La clave para lograr esta mejora es controlar el sobreajuste mediante el tamaño mínimo de hoja. Su principal ventaja es la interpretabilidad: las primeras divisiones se basan en la duración y el ROI de la película y dan lugar a reglas fáciles de explicar. Como limitación, al tener pocas hojas asigna un número reducido de probabilidades distintas, lo que restringe su capacidad para ordenar las películas. Esta limitación motiva evaluar modelos de ensamble, que combinan muchos árboles.

2.3. Regresión Logística

2.3.1. Descripción del modelo

La regresión logística estima la probabilidad de que una película reciba una opinión positiva a partir de una combinación lineal de sus atributos: a cada variable le asigna un coeficiente, y la suma ponderada se transforma con la función sigmoide en un valor entre 0 y 1. Se elige por ser un modelo simple y rápido de entrenar, cuyos coeficientes indican en qué sentido y con qué fuerza influye cada variable. Su principal limitación es que solo representa relaciones lineales entre los atributos y la opinión.

2.3.2. Modelo simple

Como punto de partida se entrena una regresión logística con los hiperparámetros por defecto de scikit-learn, es decir, regularización L2 con C = 1. El modelo alcanza un AUC de 0,815 en validación y de 0,821 en entrenamiento. A diferencia del árbol, casi no hay diferencia entre ambos valores: con solo 27 coeficientes, el modelo no tiene capacidad para memorizar los datos de entrenamiento. Además, ya supera al árbol ajustado (0,769).

2.3.3. Ajuste del modelo

Se ajustan dos hiperparámetros. El primero es C, que regula la fuerza de la regularización: cuanto menor es su valor, más se penalizan los coeficientes grandes y más simple resulta el modelo. El segundo es el tipo de regularización: L2 reduce todos los coeficientes, mientras que L1 puede llevar algunos exactamente a cero, lo que equivale a descartar variables. Mediante una búsqueda en grilla se evalúan con validación cruzada las 12 combinaciones de seis valores de C y los dos tipos de regularización.

2.3.4. Análisis

Las combinaciones se comparan con los mismos criterios utilizados para el árbol: el AUC promedio de validación, su desvío entre folds y la diferencia con el AUC de entrenamiento.

C	AUC val. con L1	AUC val. con L2	AUC train con L2
0,001	0,500 ± 0,000	0,768 ± 0,017	0,770
0,01	0,761 ± 0,014	0,799 ± 0,012	0,802
0,1	0,812 ± 0,010	0,814 ± 0,009	0,819
1	0,815 ± 0,009	**0,815 ± 0,009**	0,821
10	0,815 ± 0,009	0,815 ± 0,009	0,821
100	0,815 ± 0,009	0,815 ± 0,009	0,821

Resultados de las 12 combinaciones de hiperparámetros de la regresión logística. La fila destacada contiene la configuración elegida, que coincide con el modelo simple

Con una regularización fuerte el modelo queda demasiado simple y su rendimiento cae; en el extremo, con L1 y C = 0,001 todos los coeficientes se anulan y el modelo no distingue entre películas (AUC de 0,50). A partir de C = 0,1 los resultados se estabilizan: las ocho combinaciones restantes difieren en menos que el desvío entre folds, sin importar el tipo de regularización. A diferencia del árbol, en ningún caso aparece una brecha relevante entre entrenamiento y validación.

2.3.5. Elección

Se selecciona la configuración con C = 1 y regularización L2, que obtiene el mayor AUC de validación (0,815 ± 0,009) con una diferencia de apenas 0,005 respecto al entrenamiento. Esta configuración coincide con la del modelo simple, por lo que el ajuste de hiperparámetros no mejora el punto de partida. La siguiente tabla presenta sus métricas junto a las del árbol ajustado e incluye el AUC obtenido en Kaggle.

Modelo	Accuracy	Precision	Recall	F1	AUC val.	AUC train	AUC Kaggle
Árbol ajustado	0,697	0,705	0,669	0,685	0,769	0,813	0,771
Regresión logística	0,744	0,751	0,721	0,736	**0,815**	0,821	**0,819**

Métricas de la regresión logística elegida, comparadas con las del árbol ajustado

2.3.6. Gráficos complementarios

La matriz de confusión muestra que el modelo clasifica correctamente el 77% de las películas con opinión negativa y el 72% de las positivas, por encima del árbol en ambas clases.

Los coeficientes miden el efecto de cada variable a igualdad de las demás. La cantidad de votos, la duración y los géneros Animation y Drama aumentan la probabilidad de una opinión positiva, mientras que el género Horror, un presupuesto alto, el idioma inglés y un estreno más reciente la reducen. El ROI, segunda variable en importancia para el árbol, recibe aquí un coeficiente bajo: como se calcula a partir del presupuesto y la recaudación, su información se reparte entre los coeficientes de las tres variables. Por eso los coeficientes no deben leerse de forma aislada. Por ejemplo, el indicador de presupuesto desconocido tiene coeficiente positivo aunque esas películas reciben menos opiniones positivas que el resto: también tienen muy pocos votos, y el modelo ya las penaliza por esa vía.

Para no extender el informe, la matriz de confusión, el gráfico del efecto de la regularización y el gráfico de coeficientes se encuentran en la notebook Regresion_Logistica.ipynb, incluida en la entrega.

2.3.7. Conclusiones

La regresión logística alcanza un AUC de validación de 0,815, superior en 0,05 al del árbol ajustado, y el AUC obtenido en Kaggle (0,819) queda dentro del desvío de la validación cruzada, lo que vuelve a respaldar su uso para estimar el rendimiento sobre datos nuevos. A diferencia del árbol, el modelo no sobreajusta y el ajuste de hiperparámetros no aporta mejoras: su rendimiento no está limitado por la regularización, sino por representar únicamente relaciones lineales. Sus ventajas son la simplicidad, la estabilidad entre folds y la posibilidad de interpretar el sentido de cada coeficiente. Como limitación, no capta efectos no lineales ni interacciones entre variables, lo que motiva evaluar modelos de ensamble, que combinan muchos árboles.

3. Selección de modelo final

3.1. Comparación de modelos

3.2. Justificación de elección final

4. Componente de investigación

5. Clasificación multiclase del genero

6. Feature Engineering

6.1. Manejo de variables categóricas

6.2. Creación de nuevas características

6.3. Selección de características

6.4. Análisis del impacto de variables